SOFTWARE REQUIREMENTS SPECIFICATION

Unikly – AI-Powered Freelancing Platform

Version 1.0
Date: March 2026
Architecture: Event-Driven Microservices with Domain-Driven Design (DDD)

CONFIDENTIAL

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) serves as the single source of truth for the design, architecture, and implementation of **Unikly**, an AI-powered freelancing platform. The document defines every domain, service boundary, API contract, event flow, data model, failure strategy, and deployment configuration needed to build the system from first line of code to production deployment.

This document is intended for the sole developer implementing the project with the assistance of Claude Code for boilerplate generation, and is designed to be followed sequentially from Section 1 through to the final deployment section.

1.2 Project Vision

Unikly connects clients who need work done with freelancers who can deliver. Unlike simple job boards, the platform provides intelligent matching powered by AI, secure escrow-based payments, real-time messaging, and a hybrid notification system. The architecture is built from the ground up as an event-driven microservices system following Domain-Driven Design (DDD) principles, ensuring each service is autonomous, scalable, and resilient.

1.3 Scope

The platform encompasses the following core capabilities:

- User registration, authentication, and profile management via Keycloak
- Job posting with full lifecycle management (DRAFT through CLOSED)
- AI-powered freelancer-to-job matching with phased complexity
- Real-time messaging between clients and freelancers
- Escrow-based payment processing with Stripe integration
- Hybrid notification delivery (WebSocket, Push, Polling fallback)
- Full-text search with Elasticsearch indexing
- Observability stack with Prometheus, Grafana, and distributed tracing
- Containerized deployment via Docker Compose with service orchestration

1.4 Definitions & Acronyms

Term	Definition
DDD	Domain-Driven Design — modeling software around business domains
EDA	Event-Driven Architecture — services communicate via asynchronous events
Bounded Context	A DDD concept where each domain has its own models and logic
Aggregate	A cluster of domain objects treated as a single unit for data changes

Term	Definition
Outbox Pattern	Storing events in a DB table before publishing to ensure guaranteed delivery
Idempotency	Ensuring repeated processing of the same event produces the same result
Circuit Breaker	A fault tolerance pattern that stops calling a failing service
DLQ	Dead Letter Queue — stores events that failed processing for later retry
KRaft	Kafka Raft — Kafka’s native metadata management replacing ZooKeeper
Escrow	A financial arrangement where a third party holds payment until conditions are met

2. Technology Stack & Version Matrix

Every technology in this stack was chosen with explicit trade-off reasoning. This section documents the exact versions, the rationale for selection, and what alternatives were considered and rejected.

2.1 Complete Version Matrix

Technology	Version	Purpose
Java (OpenJDK)	21 LTS	Primary backend language; LTS ensures long-term support; virtual threads for scalability
Spring Boot	4.0.3	Microservices framework; auto-configuration, modular jars, Spring Framework 7 support
Spring Cloud Gateway	4.x (aligned)	API Gateway; rate limiting, routing, auth propagation
Spring Security	7.0.x	OAuth2 Resource Server integration with Keycloak
Spring Data JPA	4.x (aligned)	ORM for PostgreSQL with Hibernate 7
Spring WebSocket	7.0.x	Real-time bidirectional communication for messaging & notifications
Spring Kafka	4.1.x	Kafka producer/consumer integration for event-driven messaging
Resilience4j	2.3.x	Circuit breaker, retry, rate limiter, bulkhead patterns
Angular	19.x (LTS)	Frontend SPA framework; Signals, standalone components, SSR
Angular Material	19.x	UI component library; consistent design system
Keycloak	26.5.5	Identity & access management; OAuth2/OIDC provider; MCP authorization support
Apache Kafka	4.2.0	Event streaming platform; KRaft mode (no ZooKeeper); share groups
PostgreSQL	18.3	Primary relational database; ACID-compliant; per-service isolation
Elasticsearch	9.2.x	Full-text search engine; job search indexing; BM25 ranking
Redis	7.4.x	In-memory cache; session store; user presence tracking
Docker	27.x	Container runtime for all services
Docker Compose	2.x	Multi-container orchestration for local &

Technology	Version	Purpose
		staging environments
Prometheus	2.x	Metrics collection and time-series storage
Grafana	11.x	Metrics dashboards, alerting, and visualization
Stripe API	2024-12-18.acacia	Payment processing; payment intents; webhooks; transfers
Python	3.12.x	AI matching microservice (Phase 2); FastAPI runtime
FastAPI	0.115.x	Python async web framework for AI matching endpoint
Nginx	1.27.x	Reverse proxy; static asset serving for Angular build

2.2 Technology Selection Rationale

2.2.1 Why Spring Boot 4.0 Over Alternatives

Decision: Spring Boot 4.0.3 on Java 21 LTS.

Why not Quarkus? While Quarkus offers faster startup and native compilation, Spring Boot provides a significantly larger ecosystem of starters, broader community support, and mature integration with Keycloak, Kafka, and Elasticsearch. For a solo developer, the extensive documentation and Stack Overflow coverage of Spring Boot reduces development friction.

Why not Node.js/NestJS? Java provides type safety at compile time, superior multithreading with virtual threads (Project Loom), and the JVM’s garbage collector handles long-running services better than Node’s event loop under sustained load. Spring Boot’s mature transaction management is critical for payment processing.

Why Java 21 and not 25? Java 21 is the current Long-Term Support release with guaranteed updates through 2028. Java 25 is supported by Spring Boot 4 but lacks LTS stability for a production system.

2.2.2 Why Kafka Over RabbitMQ

Decision: Apache Kafka 4.2.0 in KRaft mode.

Trade-off analysis: Kafka was chosen because the platform requires high write throughput (job postings, match events, payment events), event replayability (if a consumer crashes, it can replay from the offset), and durable ordered logs per partition. RabbitMQ excels at task queuing with complex routing but does not provide log replay or native event sourcing support.

Why KRaft mode? Kafka 4.0+ removed ZooKeeper entirely. KRaft simplifies deployment by eliminating a separate coordination service, reduces operational overhead, and improves scalability. This means one fewer infrastructure component to manage.

2.2.3 Why PostgreSQL Per Service

Decision: Each microservice gets its own PostgreSQL 18.3 database instance.

Why not a shared database? A shared database creates coupling: schema changes in one service break another, query load from one service degrades another, and deployment is no longer independent. Per-service databases enforce bounded contexts at the data level, not just the code level.

Why not MongoDB? The platform's core data (payments, job contracts, user profiles) is inherently relational and requires ACID transactions. PostgreSQL's JSONB column type provides document flexibility where needed (e.g., freelancer portfolio metadata) without sacrificing transactional integrity.

2.2.4 Why Elasticsearch Alongside PostgreSQL

Decision: Elasticsearch 9.2 for search, PostgreSQL for transactional data.

Why not PostgreSQL full-text search alone? While PostgreSQL's tsvector/tsquery is capable for simple search, it lacks relevance tuning, fuzzy matching across multiple fields, faceted search, and real-time indexing at scale. Elasticsearch provides sub-100ms search across millions of job postings with BM25 relevance scoring, typo tolerance, and aggregation support for faceted filtering (by skill, budget range, location).

Sync strategy: The Search Service consumes Kafka events (JobCreatedEvent, JobUpdatedEvent) and indexes them in Elasticsearch. This is eventual consistency by design — the PostgreSQL database in Job Service remains the source of truth.

2.2.5 Why Keycloak Over Custom Auth

Decision: Keycloak 26.5.5 as the centralized identity provider.

Why not Spring Security with custom JWT? Building authentication from scratch means implementing password hashing, token rotation, refresh token management, social login, MFA, account lockout, password reset flows, and RBAC — all of which Keycloak provides out of the box. Keycloak also supports OAuth2/OIDC standards, meaning the system is interoperable with any future client or third-party integration.

Why not Auth0 or Firebase Auth? Keycloak is self-hosted, open-source, and free. For a platform handling payments and user data, controlling the identity infrastructure eliminates vendor lock-in and ensures data sovereignty.

2.2.6 Why Angular Over React

Decision: Angular 19 LTS with standalone components.

Why not React? Angular provides a batteries-included framework: built-in routing, forms, HTTP client, dependency injection, and a CLI that enforces project structure. For a solo developer, this opinionated structure prevents architectural drift. React requires assembling multiple libraries (React Router, Axios/TanStack Query, state management) which adds decision fatigue and integration risk. Angular's Signals API (stable in v19) provides fine-grained reactivity comparable to React's hooks.

2.2.7 Why Resilience4j Over Hystrix

Decision: Resilience4j 2.3.x for fault tolerance.

Why not Hystrix? Netflix Hystrix is in maintenance mode and no longer actively developed. Resilience4j is purpose-built for Java 8+ functional programming, integrates natively with Spring Boot via annotations, and provides circuit breaker, retry, rate limiter, bulkhead, and time limiter patterns — each independently configurable per service call.

3. Domain-Driven Design (DDD) Architecture

The platform is decomposed into bounded contexts, each representing an autonomous domain with its own data, logic, and API surface. Services communicate exclusively through well-defined API contracts and asynchronous events. There are no shared database tables across services.

3.1 Bounded Contexts & Domain Map

The following domains form the system's bounded contexts:

3.1.1 Core Domains (Business-Critical)

Domain	Service	Owns	Responsibility
Identity	Identity Service (Keycloak)	Auth tokens, sessions, roles	Authentication, authorization, OAuth2/OIDC, social login, MFA
User Profile	User Profile Service	User profiles, portfolios, reviews	Profile CRUD, skills management, ratings aggregation
Job Management	Job Service	Jobs, proposals, contracts	Job lifecycle (DRAFT→CLOSED), proposal management, contract creation
Matching	Matching Service	Match scores, match history	AI-powered freelancer-to-job matching, scoring, ranking
Messaging	Messaging Service	Conversations, messages	Real-time chat between client and freelancer
Payment	Payment Service	Payments, escrow, ledger	Payment intents, escrow hold/release, refunds, financial ledger

3.1.2 Supporting Domains

Domain	Service	Owns	Responsibility
Notification	Notification Service	Notification preferences, delivery log	WebSocket push, FCM push, email dispatch, notification history
Search	Search Service	Elasticsearch indexes	Full-text search, faceted filtering, search analytics
Audit/ Logging	Audit Service	Audit events, system logs	Event audit trail, compliance logging, traceId correlation

3.2 Domain Ownership Rules

Every service adheres to these non-negotiable DDD rules:

1. **Each service owns its database.** No service reads from or writes to another service's database. Data is shared only via APIs or events.

2. **Each service defines its own models.** Even if two services reference a "user," each maintains its own representation (e.g., Job Service stores clientId as a UUID, not a full User object).
3. **Communication is via API or events.** Synchronous calls use REST APIs. Asynchronous communication uses Kafka events. No shared memory, no shared queues outside Kafka.
4. **Events are facts, not commands.** An event says "JobCreated" (something happened), not "CreateMatch" (do something). Consumers decide what to do with facts.

3.3 Aggregate Design Per Domain

3.3.1 Job Aggregate (Job Service)

The Job aggregate is the central domain object. It enforces state transitions and business invariants.

Aggregate root: Job

Entities within aggregate: Proposal, Contract

Value objects: Budget (amount + currency), SkillSet, JobStatus

Invariants enforced: A job can only transition from OPEN to IN_PROGRESS if a proposal has been accepted and a contract exists. A job cannot be CANCELLED if it is IN_PROGRESS with an active escrow.

3.3.2 Payment Aggregate (Payment Service)

Aggregate root: Payment

Entities within aggregate: EscrowHold, LedgerEntry, Refund

Value objects: Money (amount + currency), PaymentStatus, ProviderReference

Invariants enforced: An escrow can only be released if the associated job is marked COMPLETED. A payment cannot be processed if an identical idempotency key already exists. Every state change produces a LedgerEntry for auditing.

3.3.3 Match Aggregate (Matching Service)

Aggregate root: MatchResult

Entities within aggregate: MatchScore, SkillOverlap

Value objects: Score (0.0–1.0), MatchStrategy (RULE_BASED, AI_EMBEDDING)

Invariants enforced: A match is only produced for jobs with status OPEN. Duplicate matches for the same job+freelancer pair are rejected.

4. Event-Driven Architecture (EDA)

All inter-service communication that does not require an immediate response uses asynchronous events published to Apache Kafka. This section defines every event topic, payload, producer, consumer, and failure strategy.

4.1 Kafka Topic Registry

Topic Name	Producer	Consumers	Partitions	Retention
job.events	Job Service	Matching, Search, Notification, Audit	6	7 days
payment.events	Payment Service	Job, Notification, Audit	4	30 days
matching.events	Matching Service	Notification, Audit	4	7 days
messaging.events	Messaging Service	Notification, Audit	6	3 days
user.events	User Profile Service	Search, Matching, Audit	4	7 days
notification.dlq	Notification Service	Audit (manual review)	2	30 days
payment.dlq	Payment Service	Audit (manual review)	2	90 days

4.2 Event Catalog (Complete Payloads)

4.2.1 Job Domain Events

JobCreatedEvent — Published when a client creates a new job posting.

Payload: { eventId: UUID, eventType: "JobCreated", timestamp: ISO-8601, jobId: UUID, clientId: UUID, title: string, description: string, skills: string[], budget: { amount: number, currency: string }, status: "OPEN" }

Consumers and reactions: Matching Service queues the job for async matching with delay. Search Service indexes the job in Elasticsearch. Notification Service alerts freelancers with matching skills. Audit Service logs the event.

JobStatusChangedEvent — Published when job status transitions.

Payload: { eventId: UUID, eventType: "JobStatusChanged", timestamp: ISO-8601, jobId: UUID, previousStatus: string, newStatus: string, triggeredBy: UUID }

ProposalSubmittedEvent — Published when a freelancer submits a proposal.

Payload: { eventId: UUID, eventType: "ProposalSubmitted", timestamp: ISO-8601, jobId: UUID, freelancerId: UUID, proposedBudget: { amount: number, currency: string }, coverLetter: string }

4.2.2 Payment Domain Events

PaymentInitiatedEvent — Published when a client starts an escrow payment.

Payload: { eventId: UUID, eventType: "PaymentInitiated", timestamp: ISO-8601, paymentId: UUID, jobId: UUID, clientId: UUID, amount: { amount: number, currency: string }, providerRef: string, status: "PENDING" }

PaymentCompletedEvent — Published when the payment provider confirms success via webhook.

Payload: { eventId: UUID, eventType: "PaymentCompleted", timestamp: ISO-8601, paymentId: UUID, jobId: UUID, providerRef: string, status: "FUNDED" }

Consumers and reactions: Job Service transitions job to FUNDED status. Notification Service alerts the freelancer that escrow is funded and work can begin. Audit Service logs the financial event.

EscrowReleasedEvent — Published when client approves work and escrow is released.

Payload: { eventId: UUID, eventType: "EscrowReleased", timestamp: ISO-8601, paymentId: UUID, jobId: UUID, freelancerId: UUID, amount: { amount: number, currency: string } }

4.2.3 Matching Domain Events

JobMatchedEvent — Published when matching completes for a job.

Payload: { eventId: UUID, eventType: "JobMatched", timestamp: ISO-8601, jobId: UUID, matches: [{ freelancerId: UUID, score: number, matchedSkills: string[] }], strategy: "RULE_BASED" | "AI_EMBEDDING" }

4.2.4 Messaging Domain Events

MessageSentEvent — Published when a message is sent in a conversation.

Payload: { eventId: UUID, eventType: "MessageSent", timestamp: ISO-8601, conversationId: UUID, senderId: UUID, recipientId: UUID, preview: string (first 100 chars) }

4.2.5 User Domain Events

UserProfileUpdatedEvent — Published when a user updates their profile or skills.

Payload: { eventId: UUID, eventType: "UserProfileUpdated", timestamp: ISO-8601, userId: UUID, updatedFields: string[], skills: string[] }

4.3 The Outbox Pattern (Guaranteed Event Delivery)

This is the most critical reliability pattern in the system. Without it, events can be lost if the application crashes between committing to the database and publishing to Kafka.

How it works:

5. Within the same database transaction that creates/updates a domain entity, an event record is inserted into an **outbox** table.
6. A background scheduler (polling every 500ms) reads PENDING events from the outbox table.
7. The scheduler publishes each event to Kafka.
8. On successful Kafka acknowledgment, the outbox record is marked as SENT.
9. If Kafka is unreachable, the event stays PENDING and is retried on the next poll cycle.

Outbox table schema (per service):

outbox_events: { id: UUID (PK), event_type: VARCHAR(100), payload: JSONB, status: ENUM(PENDING, SENT, FAILED), created_at: TIMESTAMP, sent_at: TIMESTAMP NULL, retry_count: INT DEFAULT 0 }

Why not use Kafka transactions directly? Kafka transactions require the producer and the database write to participate in a distributed transaction (2PC). This adds latency, complexity, and a single point of failure. The outbox pattern achieves guaranteed delivery using only local database transactions, which are simpler and more reliable.

4.4 Idempotent Consumer Pattern

Every event consumer must handle duplicate events safely. This is guaranteed by storing processed eventIds.

Implementation: Each consumer maintains a processed_events table: { event_id: UUID (PK), processed_at: TIMESTAMP }. Before processing any event, the consumer checks if the eventId exists. If it does, the event is silently skipped. If not, the event is processed and the eventId is recorded within the same transaction.

4.5 Dead Letter Queue (DLQ) Strategy

Events that fail processing after the maximum retry count (default: 5 with exponential backoff) are moved to a Dead Letter Queue topic.

DLQ payload wrapper: { originalEvent: {the original event}, errorMessage: string, failedAt: ISO-8601, retryCount: number, consumerService: string }

DLQ events are monitored via Grafana dashboards and trigger alerts when the DLQ depth exceeds configurable thresholds.

5. Detailed Service Specifications

5.1 API Gateway (Spring Cloud Gateway)

The gateway is the single entry point for all client requests. It handles cross-cutting concerns so that individual services do not have to.

5.1.1 Responsibilities

- **Request routing:** Routes /api/jobs/ **to Job Service, /api/payments/** to Payment Service, etc.
- **Authentication propagation:** Validates JWT tokens from Keycloak on every request. Extracts userId, roles, and permissions from the token and forwards them as headers.
- **Rate limiting:** Configurable per-client rate limits using Redis as the backing store. Default: 100 requests/minute per user, 1000 requests/minute per IP.
- **CORS configuration:** Allows requests from the Angular frontend domain only.
- **Circuit breaker:** If a downstream service is unhealthy, the gateway returns 503 instead of hanging the client connection.

5.1.2 Route Configuration

Path Pattern	Target Service	Auth Required	Rate Limit
/api/jobs/**	Job Service (8081)	Yes	100/min
/api/users/**	User Profile Service (8082)	Yes	100/min
/api/payments/**	Payment Service (8083)	Yes	50/min
/api/matches/**	Matching Service (8084)	Yes	100/min
/api/messages/**	Messaging Service (8085)	Yes	200/min
/api/ notifications/**	Notification Service (8086)	Yes	100/min
/api/search/**	Search Service (8087)	Yes	200/min
/ws/**	Notification Service (WS)	Yes (upgrade)	N/A
/webhooks/**	Payment Service (8083)	No (signature verified)	N/A

5.2 Identity Service (Keycloak)

5.2.1 Realm Configuration

A single Keycloak realm named **unikly** contains all identity configuration.

- **Clients:** unikly-frontend (public, PKCE), unikly-backend (confidential, service-account)

- **Roles:** ROLE_CLIENT, ROLE_FREELANCER, ROLE_ADMIN
- **Identity providers (optional):** Google, GitHub for social login
- **Token configuration:** Access token TTL: 5 minutes. Refresh token TTL: 30 days. Access token includes userId, email, roles, and realm_access claims.

5.2.2 Authentication Flows

Registration flow: User registers via Angular frontend → Keycloak registration page → email verification → redirect to frontend with access token. During registration, user selects role (Client or Freelancer), which is mapped to a Keycloak role.

Login flow: Angular frontend redirects to Keycloak login → Keycloak authenticates → returns authorization code → frontend exchanges code for tokens via PKCE. No client secret is exposed to the browser.

Token refresh: The Angular HTTP interceptor detects 401 responses, silently refreshes using the refresh token, and retries the original request.

5.3 User Profile Service

5.3.1 Data Model

UserProfile entity: { id: UUID (PK, same as Keycloak userId), displayName: string, bio: text, avatarUrl: string, role: ENUM(CLIENT, FREELANCER), skills: string[] (FREELANCER only), hourlyRate: decimal, currency: string, location: string, portfolioLinks: string[], averageRating: decimal, totalReviews: int, createdAt: timestamp, updatedAt: timestamp, version: int }

Review entity: { id: UUID, reviewerId: UUID, revieweeId: UUID, jobId: UUID, rating: int (1-5), comment: text, createdAt: timestamp }

5.3.2 API Endpoints

Method	Endpoint	Description	Auth
GET	/api/users/me	Get current user's profile	Bearer token
PUT	/api/users/me	Update current user's profile	Bearer token
GET	/api/users/{id}	Get any user's public profile	Bearer token
GET	/api/users?role=FREELANCER&skill=Angular&page=0&size=20	Search freelancers	Bearer token
POST	/api/users/{id}/reviews	Submit a review for a user	Bearer token + completed job
GET	/api/users/{id}/reviews?page=0&size=10	Get reviews for a user	Bearer token

5.3.3 Events Published

- **UserProfileUpdatedEvent** — on profile or skills change. Consumed by Search Service (re-index) and Matching Service (update skill cache).

5.4 Job Service

The Job Service is the most critical core domain. It manages the full job lifecycle, proposal workflow, and contract creation.

5.4.1 Job Lifecycle State Machine

Jobs follow a strict state machine with enforced transitions. The backend rejects any invalid transition attempt.

Valid transitions: DRAFT → OPEN, OPEN → IN_PROGRESS, OPEN → CANCELLED, IN_PROGRESS → COMPLETED, IN_PROGRESS → DISPUTED, COMPLETED → CLOSED, DISPUTED → COMPLETED (after resolution), DISPUTED → REFUNDED

Transition guards: OPEN → IN_PROGRESS requires an accepted proposal and a funded escrow (reacts to PaymentCompletedEvent). IN_PROGRESS → COMPLETED requires client approval. CANCELLED is only valid from OPEN and DRAFT.

5.4.2 Data Model

Job entity: { id: UUID (PK), clientId: UUID, title: VARCHAR(200), description: TEXT, budget: DECIMAL(12,2), currency: VARCHAR(3), skills: TEXT[] (PostgreSQL array), status: ENUM(DRAFT, OPEN, IN_PROGRESS, COMPLETED, CLOSED, CANCELLED, DISPUTED, REFUNDED), createdAt: TIMESTAMP, updatedAt: TIMESTAMP, version: INT }

Proposal entity: { id: UUID (PK), jobId: UUID (FK), freelancerId: UUID, proposedBudget: DECIMAL(12,2), coverLetter: TEXT, status: ENUM(PENDING, ACCEPTED, REJECTED, WITHDRAWN), createdAt: TIMESTAMP, version: INT }

Contract entity: { id: UUID (PK), jobId: UUID (FK), clientId: UUID, freelancerId: UUID, agreedBudget: DECIMAL(12,2), terms: TEXT, status: ENUM(ACTIVE, COMPLETED, TERMINATED), startedAt: TIMESTAMP, completedAt: TIMESTAMP NULL }

5.4.3 API Endpoints

Method	Endpoint	Description
POST	/api/jobs	Create a new job (status: DRAFT or OPEN based on payload)
GET	/api/jobs?status=OPEN&skill=Angular&minBudget=100&maxBudget=5000&sort=createdAt,desc&page=0&size=20	List/filter jobs with pagination
GET	/api/jobs/{id}	Get job details including proposal count
PATCH	/api/jobs/{id}	Update job details (only if DRAFT)
PATCH	/api/jobs/{id}/status	Transition job status (enforces state machine)

Method	Endpoint	Description
POST	/api/jobs/{id}/proposals	Submit a proposal (FREELANCER only)
GET	/api/jobs/{id}/proposals	List proposals for a job (JOB OWNER only)
PATCH	/api/jobs/{id}/proposals/{proposalId}/accept	Accept a proposal and create contract
PATCH	/api/jobs/{id}/proposals/{proposalId}/reject	Reject a proposal

5.4.4 Events Published

- **JobCreatedEvent** — triggers matching, search indexing, notifications
- **JobStatusChangedEvent** — triggers downstream status updates
- **ProposalSubmittedEvent** — notifies job owner
- **ProposalAcceptedEvent** — triggers contract creation, payment prompt

5.4.5 Events Consumed

- **PaymentCompletedEvent** — transitions job to FUNDED/IN_PROGRESS
- **EscrowReleasedEvent** — marks job payment as settled

5.4.6 Database Indexes

The following indexes are essential for query performance:

- idx_job_status ON jobs(status) — most queries filter by status
- idx_job_client ON jobs(client_id) — “my posted jobs” queries
- idx_job_created ON jobs(created_at DESC) — default sort order
- idx_job_budget ON jobs(budget) — budget range filtering
- idx_job_skills ON jobs USING GIN(skills) — array containment queries

5.5 Matching Service

The Matching Service consumes JobCreatedEvent asynchronously, processes matches with a configurable delay, and stores results independently. It does not depend on Job Service’s database.

5.5.1 Matching Algorithm Phases

Phase 1 (Rule-Based — implement first):

- Skill overlap: Count matching skills between job requirements and freelancer profiles. Weight: 40%.
- Budget compatibility: Compare job budget with freelancer’s hourly rate. Weight: 20%.
- Rating score: Normalize freelancer’s average rating to 0–1. Weight: 25%.
- Response time: Average time freelancer takes to respond to proposals. Weight: 15%.
- Final score = weighted sum, normalized to 0.0–1.0.

Phase 2 (AI-Enhanced — implement after Phase 1 is stable):

- Deploy a FastAPI Python microservice alongside the Java service
- Generate embeddings for job descriptions and freelancer profiles using sentence-transformers
- Compute cosine similarity between job embedding and freelancer embeddings
- Combine embedding similarity (50%) with rule-based score (50%) for hybrid ranking
- Store embeddings in PostgreSQL using pgvector extension

5.5.2 Data Model

MatchResult entity: { id: UUID (PK), jobId: UUID (indexed), freelancerId: UUID (indexed), score: DECIMAL(5,4), matchedSkills: TEXT[], strategy: ENUM(RULE_BASED, AI_EMBEDDING), createdAt: TIMESTAMP }

5.5.3 API Endpoints

Method	Endpoint	Description
GET	/api/matches?jobId={uuid}&limit=20	Get top matches for a job (sorted by score DESC)
GET	/api/matches?freelancerId={uuid} &limit=10	Get jobs matched to a freelancer
GET	/api/matches/{id}	Get specific match details

5.5.4 Async Processing Flow

1. JobCreatedEvent arrives on job.events topic
2. Matching consumer reads the event; applies configurable delay (default: 30 seconds) to batch nearby events
3. Fetches freelancer skill data from its local cache (populated by UserProfileUpdatedEvent)
4. Runs scoring algorithm, stores top N results (default: 20) in its own PostgreSQL
5. Publishes JobMatchedEvent to matching.events topic
6. If AI service (Phase 2) is unavailable, falls back to rule-based matching (graceful degradation)

5.5.5 Failure Strategy

- **Circuit breaker** on AI service calls (Resilience4j): opens after 5 failures in 60 seconds, half-open after 30 seconds
- **Fallback:** If circuit is open, use rule-based matching only
- **Retry:** 3 retries with exponential backoff (1s, 2s, 4s) for transient failures
- **DLQ:** After max retries, event moves to matching.dlq for manual investigation

5.6 Payment Service (Escrow Model)

The Payment Service manages the financial flow. It integrates with Stripe for payment processing and implements a full escrow model. **Your system is the source of truth for business state; Stripe is the source of truth for transaction status.**

5.6.1 Escrow State Machine

Payment states: PENDING → FUNDED → RELEASED → COMPLETED

Alternative flows: PENDING → FAILED, FUNDED → REFUNDED, FUNDED → DISPUTED → REFUNDED or RELEASED

5.6.2 Payment Flow (Step-by-Step)

10. **Client initiates payment:** Frontend calls POST /api/payments with jobId and amount.
11. **Payment Service creates Stripe PaymentIntent:** Calls Stripe API, receives a client_secret. Stores: { paymentId, jobId, amount, status: PENDING, stripePaymentIntentId, idempotencyKey }.
12. **Frontend completes payment:** Uses Stripe.js to collect card details and confirm the PaymentIntent. This happens entirely on Stripe's infrastructure — card numbers never touch your server.
13. **Stripe sends webhook:** POST /webhooks/stripe with event type payment_intent.succeeded. Payment Service verifies the webhook signature using Stripe's signing secret.
14. **Idempotent processing:** Check if this webhook event has already been processed (by Stripe event ID). If yes, return 200 and skip. If no, update payment status to FUNDED.
15. **Emit PaymentCompletedEvent:** Published via outbox pattern. Consumed by Job Service (mark job as funded) and Notification Service (alert freelancer).
16. **Work completion:** When client approves deliverables, frontend calls POST /api/payments/{id}/release.
17. **Escrow release:** Payment Service calls Stripe Transfer API to send funds to freelancer's connected Stripe account. Status transitions to RELEASED, then COMPLETED.
18. **Emit EscrowReleasedEvent:** Consumed by Job Service (mark job COMPLETED) and Notification Service (alert freelancer of payment).

5.6.3 Data Model

Payment entity: { id: UUID (PK), jobId: UUID, clientId: UUID, freelancerId: UUID, amount: DECIMAL(12,2), currency: VARCHAR(3), status: ENUM(PENDING, FUNDED, RELEASED, COMPLETED, FAILED, REFUNDED, DISPUTED), stripePaymentIntentId: VARCHAR(255), stripeTransferId: VARCHAR(255) NULL, idempotencyKey: VARCHAR(255) UNIQUE, createdAt: TIMESTAMP, updatedAt: TIMESTAMP, version: INT }

LedgerEntry entity: { id: UUID (PK), paymentId: UUID (FK), entryType: ENUM(ESCROW_FUND, ESCROW_RELEASE, REFUND, FEE_DEDUCTION), amount: DECIMAL(12,2), currency: VARCHAR(3), description: TEXT, createdAt: TIMESTAMP }

WebhookEvent entity (idempotency): { id: VARCHAR(255) (PK, Stripe event ID), eventType: VARCHAR(100), processedAt: TIMESTAMP }

5.6.4 API Endpoints

Method	Endpoint	Description
POST	/api/payments	Create payment intent for a job (returns Stripe client_secret)
GET	/api/payments?jobId={uuid}	Get payment status for a job
POST	/api/payments/{id}/release	Release escrow to freelancer (CLIENT only)
POST	/api/payments/{id}/refund	Request refund (before release only)
POST	/webhooks/stripe	Stripe webhook endpoint (no auth, signature-verified)

5.6.5 Crash Recovery Scenarios

Scenario 1: System crashes after Stripe charges card but before saving to DB. Stripe retries the webhook. On restart, the webhook arrives again, is processed idempotently, and the payment record is created.

Scenario 2: Payment saved to DB but event not published to Kafka. The outbox pattern handles this. The event remains PENDING in the outbox table and is published on the next poll cycle.

Scenario 3: Duplicate webhook from Stripe. The WebhookEvent table's primary key (Stripe event ID) ensures the event is processed exactly once.

Scenario 4: User closes browser mid-payment. Does not matter. The Stripe PaymentIntent persists on Stripe's side. When the user returns, the frontend can resume using the same client_secret, or Stripe sends the webhook regardless.

5.7 Messaging Service

5.7.1 Data Model

Conversation entity: { id: UUID (PK), jobId: UUID (nullable), participantIds: UUID[], createdAt: TIMESTAMP, lastMessageAt: TIMESTAMP }

Message entity: { id: UUID (PK), conversationId: UUID (FK), senderId: UUID, content: TEXT (encrypted at rest), contentType: ENUM(TEXT, FILE_LINK, SYSTEM), readAt: TIMESTAMP NULL, createdAt: TIMESTAMP }

5.7.2 Real-Time Delivery

Messages are delivered in real-time via WebSocket. When a message is sent via the REST API, the service simultaneously persists it and pushes it to the recipient's WebSocket connection (if online). A MessageSentEvent is also published for the Notification Service to handle offline delivery.

5.7.3 API Endpoints

Method	Endpoint	Description
GET	/api/messages/conversations?page=0&size=20	List user's conversations
GET	/api/messages/conversations/{id}?	Get messages in a conversation

Method	Endpoint	Description
	page=0&size=50	
POST	/api/messages/conversations/{id}	Send a message
PATCH	/api/messages/{messageId}/read	Mark message as read
WS	/ws/messages	WebSocket endpoint for real-time message delivery

5.8 Notification Service

This is a cross-cutting service that consumes events from all other services and handles delivery through multiple channels.

5.8.1 Hybrid Delivery Strategy

The service uses a three-tier delivery strategy with automatic fallback:

19. **WebSocket (primary):** If the user has an active WebSocket connection (tracked via Redis presence), push the notification instantly.
20. **Push notification (secondary):** If the user is offline but has registered a device token, send via Firebase Cloud Messaging (FCM).
21. **Polling fallback (tertiary):** The frontend polls GET /api/notifications?unread=true every 30 seconds if WebSocket connection fails.

5.8.2 User Presence Tracking

Redis stores user presence: key = user:{userId}:online, value = WebSocket session ID, TTL = 120 seconds (refreshed on heartbeat). When a notification event arrives, the service checks Redis. If the key exists, deliver via WebSocket. Otherwise, fall back to push.

5.8.3 Notification Data Model

Notification entity: { id: UUID (PK), userId: UUID, type: ENUM(JOB_MATCHED, PROPOSAL_RECEIVED, PAYMENT_FUNDED, ESCROW_RELEASED, MESSAGE_RECEIVED, SYSTEM), title: VARCHAR(200), body: TEXT, read: BOOLEAN DEFAULT FALSE, actionUrl: VARCHAR(500) NULL, createdAt: TIMESTAMP }

5.8.4 Notification Preferences

NotificationPreference entity: { userId: UUID (PK), emailEnabled: BOOLEAN, pushEnabled: BOOLEAN, realtimeEnabled: BOOLEAN, quietHoursStart: TIME NULL, quietHoursEnd: TIME NULL }

5.8.5 API Endpoints

Method	Endpoint	Description
GET	/api/notifications?unread=true&page=0&size=20	Get notifications (polling fallback)
PATCH	/api/notifications/{id}/read	Mark notification as read
PATCH	/api/notifications/read-all	Mark all notifications as read

Method	Endpoint	Description
GET	/api/notifications/preferences	Get notification preferences
PUT	/api/notifications/preferences	Update notification preferences
WS	/ws/notifications	WebSocket for real-time notification push

5.9 Search Service

5.9.1 Elasticsearch Index Design

Index: jobs — Maps to the Job entity but optimized for search.

Fields: { jobId: keyword, title: text (analyzed, BM25), description: text (analyzed, BM25), skills: keyword[] (exact match + aggregation), budget: float (range queries), currency: keyword, status: keyword, clientId: keyword, createdAt: date }

Index: freelancers — Maps to freelancer profiles.

Fields: { userId: keyword, displayName: text, skills: keyword[], hourlyRate: float, averageRating: float, location: text, bio: text (analyzed) }

5.9.2 API Endpoints

Method	Endpoint	Description
GET	/api/search/jobs?q=angular dashboard&skills=Angular,TypeScript&minBudget=100&maxBudget=5000&page=0&size=20	Full-text job search with filters
GET	/api/search/freelancers?q=java developer&skills=Java,Spring&minRating=4&page=0&size=20	Full-text freelancer search
GET	/api/search/suggestions?q=ang&type=skill	Autocomplete suggestions

5.9.3 Sync Strategy

The Search Service consumes Kafka events to keep its indexes in sync:

- JobCreatedEvent → Index new job document
- JobStatusChangedEvent → Update status field (or remove from index if CANCELLED/CLOSED)
- UserProfileUpdatedEvent → Re-index freelancer document

Consistency model: Eventual consistency with sub-second lag under normal conditions. Elasticsearch is NOT the source of truth — always fall back to service APIs for authoritative data.

6. Resilience & Observability

6.1 Resilience Patterns (Per Service)

Pattern	Library	Configuration	Where Applied
Circuit Breaker	Resilience 4j	failureRateThreshold: 50%, waitDurationInOpenState: 30s, slidingWindowSize: 10	AI service calls, Stripe API calls, inter-service REST calls
Retry	Resilience 4j	maxAttempts: 3, waitDuration: 1s, exponentialBackoff: true, multiplier: 2	Kafka publish failures, Stripe webhook processing, Elasticsearch indexing
Rate Limiter	Resilience 4j	limitForPeriod: 50, limitRefreshPeriod: 1s	API Gateway per-client limits, Stripe API calls
Timeout	Resilience 4j	timeoutDuration: 5s (REST), 30s (AI matching)	All inter-service REST calls, external API calls
Bulkhead	Resilience 4j	maxConcurrentCalls: 25, maxWaitDuration: 500ms	Payment processing, AI matching computation

6.2 Fallback Behaviors

Service Down	Fallback Behavior
AI Matching Service (Python)	Fall back to rule-based matching (Java). User receives matches within 2 minutes instead of 30 seconds.
Elasticsearch	Search Service returns empty results with 503 status. Frontend displays "Search temporarily unavailable" and prompts user to browse by category.
Stripe API	Payment Service returns 503. Frontend shows "Payment processing temporarily unavailable, please try again in a few minutes." Retry with backoff.
Notification Service	Events accumulate in Kafka. On service recovery, notifications are delivered in order. No data loss.
Redis (presence tracking)	Notification Service defaults to push notification for all users (assumes offline). Slightly higher FCM usage but no missed notifications.

6.3 Observability Stack

6.3.1 Metrics (Prometheus + Grafana)

Every Spring Boot service exposes metrics via Spring Boot Actuator at /actuator/prometheus. Prometheus scrapes all services every 15 seconds.

Key metrics per service:

- http_server_requests_seconds (latency histogram, tagged by path, method, status)
- kafka_consumer_records_consumed_total (events processed)
- kafka_consumer_records_lag (consumer lag — critical for detecting processing delays)
- resilience4j_circuitbreaker_state (CLOSED, OPEN, HALF_OPEN per named breaker)
- jvm_memory_used_bytes, jvm_threads_live_threads, jvm_gc_pause_seconds
- Custom: job_created_total, payment_completed_total, match_generated_total, match_duration_seconds

6.3.2 Distributed Tracing

All services propagate a traceId (via Micrometer Tracing + OpenTelemetry) through HTTP headers and Kafka message headers. A single request from the frontend generates one traceId that flows through Gateway → Job Service → Kafka → Matching Service → Notification Service. This enables end-to-end request tracing in Grafana Tempo or Jaeger.

6.3.3 Logging

All services use structured JSON logging (Logback with JSON encoder). Every log line includes: timestamp, level, service name, traceId, spanId, userId (from JWT), and the message. Log aggregation is via Grafana Loki or ELK stack.

6.3.4 Alerting Rules

Alert	Condition	Severity
High API Latency	p95 response time > 2s for 5 minutes	Warning
Circuit Breaker Open	Any circuit breaker in OPEN state for > 2 minutes	Critical
Kafka Consumer Lag	Consumer lag > 1000 messages for > 5 minutes	Warning
DLQ Depth	Any DLQ topic has > 10 messages	Critical
Payment Failure Rate	Payment failure rate > 5% over 15 minutes	Critical
Service Down	Health check fails for > 30 seconds	Critical

7. Security Architecture

7.1 Authentication & Authorization Flow

All authentication is delegated to Keycloak. Services never handle passwords directly.

22. Frontend obtains JWT access token from Keycloak via Authorization Code + PKCE flow.
23. Every API request includes the JWT in the Authorization: Bearer <token> header.
24. API Gateway validates the JWT signature against Keycloak's public key (cached, refreshed hourly).
25. Gateway extracts userId, roles, and email from token claims and forwards them as X-User-Id, X-User-Roles headers to downstream services.
26. Each service enforces role-based access: only ROLE_CLIENT can post jobs, only ROLE_FREELANCER can submit proposals, only ROLE_ADMIN can access admin endpoints.

7.2 Service-to-Service Security

Internal services communicate within a Docker network and are not exposed to the public internet. The API Gateway is the only publicly exposed endpoint. For additional security, inter-service calls use the Keycloak service account (client_credentials grant) to obtain machine-to-machine tokens.

7.3 Data Security

- **Encryption at rest:** PostgreSQL databases use transparent data encryption (TDE) or volume-level encryption.
- **Encryption in transit:** All HTTP communication uses TLS 1.3. Kafka brokers use SSL for inter-broker and client communication.
- **PII handling:** User email, phone, and payment info are classified as PII. Message content is encrypted at rest in the database.
- **Stripe webhook verification:** Every incoming webhook is verified using Stripe's webhook signing secret via HMAC-SHA256. Unverified webhooks are rejected with 400.

7.4 Input Validation & Protection

- All API inputs are validated via Jakarta Bean Validation annotations (@NotBlank, @Size, @Min, @Max, @Pattern).
- SQL injection prevention via parameterized queries (JPA/Hibernate).
- XSS prevention via Angular's built-in sanitization and Content-Security-Policy headers.
- CSRF protection via SameSite cookies and CSRF tokens for state-changing operations.
- Rate limiting at the gateway prevents brute-force and DDoS attempts.

8. Frontend Architecture (Angular)

8.1 Project Structure

The Angular app follows a feature-module architecture with standalone components (no NgModules).

src/app/ structure: core/ (guards, interceptors, services shared globally), features/ (auth/, jobs/, proposals/, messaging/, payments/, profile/, notifications/, search/, admin/), shared/ (UI components, pipes, directives), layouts/ (main-layout, auth-layout).

8.2 State Management

Angular Signals are the primary state management mechanism for local component state. For shared state (current user, notification count, WebSocket connection), a lightweight service-based store using Angular Signals is used. No external state library (NgRx) is needed for this project scope.

8.3 Key Frontend Flows

8.3.1 Job Posting Flow

27. Client navigates to /jobs/create.
28. Fills in title, description, selects skills from autocomplete (calls /api/search/suggestions), sets budget.
29. Submits form. Angular HTTP client sends POST /api/jobs with JWT in Authorization header.
30. On success, redirects to /jobs/{id} with toast notification “Job posted successfully.”
31. Within 30–60 seconds, WebSocket notification arrives: “We found N freelancers matching your job.”

8.3.2 Payment Flow

32. Client accepts a proposal. Frontend prompts to fund escrow.
33. Frontend calls POST /api/payments. Backend returns Stripe client_secret.
34. Frontend loads Stripe.js, renders the payment element, collects card info.
35. User confirms. Stripe.js sends card data directly to Stripe (PCI-compliant; card data never touches your server).
36. On success, frontend shows “Payment processing...” and waits for WebSocket notification confirming escrow is funded.

8.3.3 Real-Time Messaging Flow

37. User navigates to /messages. Frontend opens WebSocket to /ws/messages with JWT as query param.
38. Existing conversations are loaded via GET /api/messages/conversations.
39. User selects a conversation. Messages are loaded via REST with pagination (scroll-up loads older messages).
40. User types and sends a message via POST /api/messages/conversations/{id}.

41. Server persists the message and pushes it to the recipient's WebSocket in real-time.
42. Typing indicators are sent via WebSocket (ephemeral, not persisted).

8.4 Angular HTTP Interceptor Chain

- **AuthInterceptor:** Attaches JWT access token to every outgoing request. On 401, attempts silent token refresh via Keycloak's refresh token. If refresh fails, redirects to login.
- **ErrorInterceptor:** Catches HTTP errors, displays user-friendly toast notifications, and logs errors to console. 503 errors trigger "Service temporarily unavailable" message.
- **LoadingInterceptor:** Shows/hides a global loading spinner for long-running requests (configurable threshold: 500ms).

9. Docker Compose & Deployment Architecture

9.1 Service Container Map

Container	Image	Exposed Port	Depends On
nginx	nginx:1.27	80, 443	gateway
gateway	Custom (Spring Boot)	8080	keycloak, redis
keycloak	quay.io/keycloak/keycloak:26.5.5	8180	postgres-keycloak
job-service	Custom (Spring Boot)	8081	postgres-jobs, kafka
user-service	Custom (Spring Boot)	8082	postgres-users, kafka
payment-service	Custom (Spring Boot)	8083	postgres-payments, kafka
matching-service	Custom (Spring Boot)	8084	postgres-matching, kafka, redis
messaging-service	Custom (Spring Boot)	8085	postgres-messaging, kafka, redis
notification-service	Custom (Spring Boot)	8086	postgres-notifications, kafka, redis
search-service	Custom (Spring Boot)	8087	elasticsearch, kafka
matching-ai	Custom (Python/FastAPI)	8090	(standalone)
kafka	apache/kafka:4.2.0	9092	(KRaft, no ZooKeeper)
postgres-jobs	postgres:18	5433	(data volume)
postgres-users	postgres:18	5434	(data volume)
postgres-payments	postgres:18	5435	(data volume)
postgres-matching	postgres:18	5436	(data volume)
postgres-messaging	postgres:18	5437	(data volume)
postgres-notifications	postgres:18	5438	(data volume)
postgres-keycloak	postgres:18	5439	(data volume)
elasticsearch	elasticsearch:9.2.0	9200	(data volume)
redis	redis:7.4	6379	(data volume)
prometheus	prom/prometheus:latest	9090	(all services)
grafana	grafana/grafana:latest	3000	prometheus

9.2 Network Architecture

All containers run on a single Docker bridge network named **unikly-net**. Services reference each other by container name (e.g., kafka:9092, postgres-jobs:5432). Only nginx (port 80/443) is exposed to the host. All other ports are internal to the Docker network.

9.3 Volume Strategy

Each PostgreSQL instance has a named Docker volume for data persistence: unikly-pg-jobs-data, unikly-pg-users-data, etc. Elasticsearch gets unikly-es-data. Kafka gets unikly-kafka-data. Volumes survive container restarts and rebuilds.

9.4 Environment Configuration

Each service reads configuration from environment variables defined in the Docker Compose file. Sensitive values (database passwords, Stripe keys, Keycloak admin credentials) are stored in a .env file that is NOT committed to version control. The .env.example file documents all required variables.

9.5 Health Checks

Every service container defines a Docker health check:

- Spring Boot services: curl -f http://localhost:{port}/actuator/health
- PostgreSQL: pg_isready -U postgres
- Kafka: kafka-broker-api-versions.sh --bootstrap-server localhost:9092
- Elasticsearch: curl -f http://localhost:9200/_cluster/health
- Redis: redis-cli ping
- Keycloak: curl -f http://localhost:8080/health/ready

10. Implementation Roadmap

This roadmap is designed for a solo developer using Claude Code for boilerplate acceleration. Each phase builds on the previous one and is independently testable.

Phase 1: Foundation (Weeks 1–2)

Goal: Infrastructure running, one service deployed, auth working end-to-end.

43. Set up monorepo structure: /backend (Gradle multi-module), /frontend (Angular), /infra (Docker Compose, configs)
44. Create Docker Compose with: PostgreSQL (single instance for dev), Kafka (KRaft), Keycloak, Redis, Elasticsearch
45. Configure Keycloak: create realm, clients, roles (CLIENT, FREELANCER, ADMIN)
46. Implement API Gateway with Keycloak JWT validation and route configuration
47. Scaffold Angular app with auth module: login, register, token interceptor
48. Verify end-to-end: Angular → Keycloak login → JWT → Gateway → protected endpoint returns 200

Phase 2: Core Services (Weeks 3–5)

Goal: Job posting, profile management, and proposals working.

49. Implement User Profile Service: CRUD, skills, reviews
50. Implement Job Service: full lifecycle state machine, proposals, contracts
51. Set up Kafka topics and implement outbox pattern in Job Service
52. Build Angular pages: job list, job detail, job creation form, profile page
53. Implement Search Service: consume events, index in Elasticsearch, expose search API
54. Build Angular search page with filters and pagination

Phase 3: Matching & Notifications (Weeks 6–7)

Goal: Intelligent matching and real-time notifications operational.

55. Implement Matching Service (Phase 1: rule-based matching)
56. Implement Notification Service with WebSocket delivery
57. Build notification bell UI in Angular with real-time updates
58. Add notification preferences (email, push, real-time toggles)
59. Implement circuit breaker and fallback patterns across all services

Phase 4: Payments & Messaging (Weeks 8–10)

Goal: Full payment flow and real-time chat working.

60. Implement Payment Service with Stripe integration (test mode)
61. Build escrow flow: fund, release, refund
62. Implement webhook endpoint with idempotent processing

63. Build Angular payment UI using Stripe.js Elements
64. Implement Messaging Service with WebSocket real-time delivery
65. Build Angular chat interface with conversation list and message thread

Phase 5: Polish & Observability (Weeks 11–12)

Goal: Production-ready with monitoring, error handling, and documentation.

66. Add Prometheus metrics to all services, configure Grafana dashboards
67. Set up alerting rules for critical conditions
68. Implement distributed tracing with Micrometer + OpenTelemetry
69. Add comprehensive error handling and user-facing error messages
70. Split Docker Compose into per-service PostgreSQL databases (production config)
71. Write OpenAPI specs for all services (SpringDoc/Swagger)
72. Create README with setup instructions, architecture diagrams, and demo walkthrough

Phase 6: AI Enhancement (Weeks 13–14, Optional)

Goal: Upgrade matching to AI-powered embeddings.

73. Deploy FastAPI Python service with sentence-transformers
74. Implement embedding generation and cosine similarity matching
75. Add pgvector extension to Matching Service’s PostgreSQL
76. Implement hybrid scoring: 50% embedding similarity + 50% rule-based
77. Add circuit breaker on AI service with fallback to rule-based matching

11. Appendix

11.1 Project Directory Structure

```
unikly/
├── backend/ (Gradle multi-module project)
│   ├── gateway/ (Spring Cloud Gateway)
│   ├── job-service/
│   ├── user-service/
│   ├── payment-service/
│   ├── matching-service/
│   ├── messaging-service/
│   ├── notification-service/
│   ├── search-service/
│   └── common/ (shared DTOs, event classes, outbox base)
├── frontend/ (Angular 19 workspace)
├── matching-ai/ (Python FastAPI service)
├── infra/
│   ├── docker-compose.yml
│   ├── docker-compose.prod.yml
│   ├── keycloak/ (realm export JSON)
│   ├── prometheus/ (prometheus.yml)
│   ├── grafana/ (provisioned dashboards)
│   └── nginx/ (nginx.conf)
├── .env.example
└── README.md
```

11.2 Coding Conventions (For Claude Code)

When using Claude Code to generate boilerplate, enforce these conventions:

- **Package structure per service:** com.unikly.{service}.domain, com.unikly.{service}.application, com.unikly.{service}.infrastructure, com.unikly.{service}.api
- **Domain layer:** Entities, value objects, domain events, repository interfaces (no framework imports)
- **Application layer:** Use case services, event handlers, DTOs, mappers
- **Infrastructure layer:** JPA repository implementations, Kafka producers/consumers, external API clients
- **API layer:** REST controllers, request/response DTOs, exception handlers

- **Naming:** Services end in Service, repositories end in Repository, controllers end in Controller, events end in Event, DTOs use Request/Response suffix
- **Testing:** Unit tests for domain logic, integration tests for API endpoints (TestContainers for PostgreSQL and Kafka), contract tests for events

11.3 Environment Variables Reference

Variable	Service	Example Value
DB_HOST	All services	postgres-jobs
DB_PORT	All services	5432
DB_NAME	Per service	unikly_jobs
DB_USERNAME	All services	unikly
DB_PASSWORD	All services	(secret)
KAFKA_BOOTSTRAP_SERVERS	All services	kafka:9092
KEYCLOAK_URL	Gateway, services	http://keycloak:8080
KEYCLOAK_REALM	Gateway, services	unikly
STRIPE_SECRET_KEY	Payment Service	sk_test_...
STRIPE_WEBHOOK_SECRET	Payment Service	whsec_...
STRIPE_PUBLISHABLE_KEY	Frontend	pk_test_...
REDIS_HOST	Gateway, Notification, Matching	redis
REDIS_PORT	Gateway, Notification, Matching	6379
ELASTICSEARCH_URL	Search Service	http://elasticsearch:9200
FCM_SERVER_KEY	Notification Service	(Firebase key)
AI_SERVICE_URL	Matching Service	http://matching-ai:8090

— End of Document —